`__VA_OPT__` wording clarifications

Document number:	P1042R0
Date:	2018-04-27
Project:	ISO/IEC JTC 1/SC 22/WG 21/C++
Audience subgroup:	Core
Revises:	*None*

Reply-to: Hubert S.K. Tong <hubert.reinterpretcast@gmail.com>

Background and Context

A number of clarifications to the adopted wording were found necessary in private correspondence conducted among Thomas Köppe, Richard Smith, Hubert Tong, and Faisal Vali. Inquiries were also made on the WG 14 and Core reflectors by Jens Gustedt and Jason Merrill. This paper clarifies the relationship between `__VA_OPT__` and substitution, stringization, and concatenation; especially with regards to placemarker tokens and white space.

Wording

The editing instructions within this document use N4741 as its base wording.

☐ Hide inserted text ☐ Hide deleted text

In subclause 19.3.1 [cpp.subst] paragraph 1, modify:

> After the arguments for the invocation of a function-like macro have been identified, argument substitution takes place. A parameter in the replacement list~~,~~ unless it is `__VA_OPT__(`*content*`)`, preceded by a `#` or `##` preprocessing token, or followed by a `##` preprocessing token (see below)~~,~~ is replaced by the corresponding argument after all macros contained therein have been expanded~~.~~ ~~Before being substituted~~, in which case, each argument's preprocessing tokens are completely macro replaced before being substituted as if they formed the rest of the preprocessing file~~;~~ with no other preprocessing tokens ~~are~~ being available. The replacement for `__VA_OPT__(`*content*`)`, unless preceded by a `#` or `##` preprocessing token or followed by a `##` preprocessing token, is the preprocessing token sequence for the corresponding argument.

Drafting note: The above clarifies that `__VA_OPT__` expansions are not rescanned prior to rescanning of the replacement list containing the instance of `__VA_OPT__`.

Add an example to [cpp.subst] paragraph 1:

```
#define RPAREN() (
#define G(Q) 42
#define F(R, X, ...)  __VA_OPT__(G R X) )
int x = F(RPAREN(), 0, <:-); // replaced by int x = 42;
```

In subclause 19.3.1 [cpp.subst] paragraph 3, modify:

> The identifier `__VA_OPT__` shall always occur as part of the preprocessing token sequence `__VA_OPT__(`*content*`)`, […]

In subclause 19.3.1 [cpp.subst] paragraph 3, modify:

> […] The preprocessing token sequence `__VA_OPT__(`*content*`)` shall be treated as if it were a parameter, and the preprocessing ~~tokens used to replace it are~~ token sequence for the corresponding argument is defined as follows. If the ~~variable arguments consist~~ substitution of `__VA_ARGS__` as neither an operand of `#` nor `##` consists of no preprocessing tokens, the ~~replacement~~ argument consists of a single placemarker preprocessing token. Otherwise, the ~~replacement~~ argument consists of the results of the expansion of *content* as the replacement list of the current function-like macro before removal of placemarker tokens, rescanning, and further replacement. [*Note:* The placemarker tokens are removed before stringization (19.3.2 [cpp.stringize]), and can be removed by rescanning and further replacement (19.3.4 [cpp.rescan]). *—end note*]

Add the following examples to 19.3.1 [cpp.subst] paragraph 3:

```
#define F(...) f(0 __VA_OPT__(,) __VA_ARGS__)
#define EMP
F(EMP) // replaced by f(0)
```

Drafting note: The above was requested on the WG 14 reflector and differs from what current implementations do.

```
#define H3(X, ...) #__VA_OPT__(X##X X##X)
H3(, 0) // replaced by ""
```

Drafting note: Clang replaces `H3` as presented.

```
#define H4(X, ...) __VA_OPT__(a X ## X) ## b
H4(, 1) // replaced by a b
```

Drafting note: The `H4` case differs from what current implementations do. The removal of placemarkers was found to give surprising results for concatenation.

```
#define H5A(...) __VA_OPT__()/**/__VA_OPT__()
#define H5B(X) a ## X ## b
#define H5C(X) H5B(X)
H5C(H5A()) // replaced by ab
```

In subclause 19.3.2 [cpp.stringize] paragraph 2, modify:

> […] If, in the replacement list, a parameter is immediately preceded by a # preprocessing token, both are replaced by a single character string literal preprocessing token that contains the spelling of the preprocessing token sequence for the corresponding argument (excluding placemarker tokens). Let the *stringizing argument* be the preprocessing token sequence for the corresponding argument with placemarker tokens removed. Each occurrence of white space between the stringizing argument's preprocessing tokens becomes a single space character in the character string literal. White space before the first preprocessing token and after the last preprocessing token comprising the stringizing argument is deleted. Otherwise, the original spelling of each preprocessing token in the stringizing argument is retained in the character string literal, except for special handling for producing the spelling of string literals and character literals: a \ character is inserted before each " and \ character of a character literal or string literal (including the delimiting " characters). If the replacement that results is not a valid character string literal, the behavior is undefined. The character string literal corresponding to an empty stringizing argument is "". […]